Arab Open University - Bahrain

HyperTool: A Comprehensive Cybersecurity Desktop Application

Senior Project Report

Student Name: [Naseem Ibrahim Khlifat]

Student ID: [6240209]

Module Code: TM471-A

Supervisor: [Dr. Salabat Khan]

Abstract

Most cybersecurity tools available today are either expensive or too hard to use for normal people. This was the main problem that HyperTool was created to solve. The idea was to build one desktop application that brings the most important security features together in a simple and clean interface. These features include link scanning, file malware detection, removing metadata from images, and a password manager. There is also an AI assistant that can answer questions related to security.

The application was built using two main technologies. React and TypeScript were used to create the front-end, which is what the user sees and interacts with. On the other hand, Rust was used for the back-end processing. The reason for this combination is that it keeps the application running fast and also makes sure that local data is stored safely. For protecting sensitive data like passwords, the system uses encryption standards that are well known in the industry, such as AES 256 GCM and Argon2id.

Another thing that was important during the development was making security feel less boring for users. Because of that, a gamification system was added where users earn XP and achievements when they use the different tools. This report covers the whole process of building HyperTool, starting from the initial requirements and design decisions, all the way to testing and evaluating the final product.

Acknowledgements

There are many people who helped me finish this project, and I would like to thank them. First, I want to thank my supervisor Dr. Salabat Khan for his guidance and support during the whole project. He always gave me useful feedback and helped me stay on the right track.

I also want to thank the academic staff at Arab Open University, especially the teachers of the software engineering, cybersecurity, and web development courses. What I learned from these courses gave me a strong base to work on this project.

In addition, I would like to thank the developers of the open-source tools that I used in this project, including the Tauri community, the React team, and the Rust developers. Without their work, building this application would not have been possible.

Finally, I want to thank my family and friends for being patient and supporting me during this project. It was not always easy, but their encouragement made a big difference.

List of Tables

Table 1.1: Project Schedule and Milestones

Table 2.1: Comparison of Existing Cybersecurity Tools

Table 3.1: Functional Requirements Summary

Table 3.2: Non-Functional Requirements Summary

Table 3.3: Hardware Requirements

Table 3.4: Software Requirements

Table 4.1: Comparison of Development Models

Table 4.2: Database Collections and Documents

Table 4.3: Testing Strategy Overview

Table 4.4: Test Case Results Summary

Table 5.1: Objectives Achievement Analysis

List of Figures :

Figure 2.1: Market Trends in Cybersecurity Applications

Figure 3.1: Use Case Diagram

Figure 3.2: Activity Diagram - URL and File Scanning

Figure 3.3: Entity-Relationship Diagram

Figure 4.1: System Architecture Diagram

Figure 4.2: Component Diagram

Figure 4.3: Data Flow Diagram - Level 0

Figure 4.4: Data Flow Diagram - Level 1

Figure 4.5: Dashboard Interface Design

Figure 5.1: Security Score Components

Figure 5.2: Level Progression Thresholds

Glossary

AES-GCM: Advanced Encryption Standard - Galois/Counter Mode, a symmetric key encryption algorithm

Argon2id:A password hashing algorithm resistant to side-channel attacks and GPU cracking

API: Application Programming Interface

CBC: Cipher Block Chaining, a block cipher mode of operation

EXIF: Exchangeable Image File Format, metadata stored in images

Firebase: Google’s mobile and web application development platform

Firestore: Firebase's NoSQL cloud database

Gamification: Application of game-design elements in non-game contexts

OAuth: Open Authorization, a standard for access delegation

Tauri: A framework for building desktop applications with web technologies

VirusTotal: An online service that analyzes files and URLs for malware

XP: Experience Points, used in the gamification system

ZTA: Zero Trust Architecture

Table of Contents

Abstract **2**

Acknowledgements **3**

List of Tables **4**

List of Figures : **5**

Glossary **6**

Table of Contents **7**

Chapter 1: Introduction **12**

1.1. Introduction 12

1.2. Background of the Study 12

1.3. Project Aims 13

1.4. Project Objectives 14

1.4.1. General Objectives 14

1.4.2. Specific Objectives 14

1.5. Problem Definition 15

1.5.1. Problem 1: Fragmented Security Tools 16

1.5.2. Problem 2: Image Privacy Risks 17

1.5.3. Problem 3: Password Management Challenges 17

1.5.4. Problem 4: Lack of Accessible Security Guidance 17

1.5.5. Problem 5: Complex Enterprise Solutions 17

1.6. Suggested Solution 18

1.6.1. Unified Security Platform 18

1.6.2. Privacy-First Design 18

1.6.3. Integrated Password Management 19

1.6.4. AI-Powered Assistance 19

1.6.5. Gamification for Engagement 19

1.6.6. Cross-Platform Desktop Application 20

1.7. Targeted Customers or Beneficiaries 20

1.7.1. Individual Consumers 20
1.7.2. Small Business Owners 20
1.7.3. Students and Academics 20
1.7.4. Remote Workers 20
1.7.5. Privacy Advocates 20
1.8. Project Scope/Delimitation 21
1.8.1. In Scope 21
1.8.2. Out of Scope 21
1.8.3. Delimitations 21
1.9. Project Schedule 22
1.10. Summary of Chapters 23
Chapter 2: Literature Review 24
2.1. Introduction 24
2.2. Search Strategy Used 24
2.2.1. Academic Databases 24
2.2.2. Industry Sources 24
2.2.3. Technical Documentation 24
2.2.4. Search Terms 24
2.3. Review of Related Literature 25
2.3.1. Unified Security Platforms 25
2.3.2. Password Management 25
2.3.3. Threat Detection and Malware Scanning 26
2.3.4. Image Privacy and Metadata Removal 26
2.3.5. Gamification in Security 27
2.3.6. Technology Stack Analysis 28
2.4. Summary of the Literature Review 30
2.5. Compare and Contrast (Features of Similar Systems and Proposed System) 30
Chapter 3: Requirements and Analysis 32
3.1. Introduction 32

3.2. Software Development Model Used 32
3.2.1. Adaptability 32
3.2.2. Early Delivery 32
3.2.3. Risk Management 32
3.3. Requirement Elicitation 32
3.3.1. Market Analysis 33
3.3.2. Academic Research 33
3.3.3. User Personas 33
3.3.4. Technical Feasibility 33
3.4. Functional Requirements 33
3.5. Non-Functional Requirements 35
3.6. Software Requirements 36
3.6.1. Frontend Development 36
3.6.2. Backend Development 36
3.6.3. External Services 36
3.7. Hardware Requirements 37
3.8. Software Modelling Tools 37
3.9. Analysis Diagrams 37
3.9.1. Use Case Diagram 37
3.9.2. Activity Diagram 39
3.9.3. Entity-Relationship Diagram 40
Chapter 4: Design, Implementation and Testing 42
4.1. Introduction 42
4.2. The Software Development Model Used 42
4.3. Justification of the Model Selected 43
4.3.1. Flexibility 43
4.3.3. User Feedback 43
4.3.4. Iterative Improvement 43
4.4. Advantages of the Selected Approach 43

4.5. Alternative Model/Approach 44
4.6. Justification for Not Selecting the Alternative Approach 44
4.6.1. Inflexibility 44
4.6.2. Late Testing 44
4.6.3. Delayed Feedback 44
4.6.4. Risk Accumulation 44
4.7. Database Design 45
4.8. User Interface Design 45
4.8.1. Layout 46
4.8.2. Color Scheme 46
4.8.3. Typography 46
4.8.4. Components 46
4.8.5. Responsiveness 46
4.9. Design Diagrams 47
4.9.1. System Architecture 47
4.9.2. Component Diagram 47
4.9.3. Data Flow Diagram - Level 0 48
4.9.4. Data Flow Diagram - Level 1 49
4.9.5. Dashboard Interface Design 50
4.9.6. Encryption Module Flow 51
4.10. Implementation ..... **Error! Bookmark not defined.**
4.10.1. Frontend Implementation ..... **Error! Bookmark not defined.**
4.10.2. Backend Implementation ..... **Error! Bookmark not defined.**
4.10.3. Key Implementation Highlights ..... **Error! Bookmark not defined.**
4.11. Testing ..... **Error! Bookmark not defined.**
4.11.1. Testing Strategy ..... **Error! Bookmark not defined.**
4.11.2. Functional Testing ..... **Error! Bookmark not defined.**
4.11.3. Non-Functional Testing ..... **Error! Bookmark not defined.**
Chapter 5: Results and Discussion ..... Error! Bookmark not defined.

5.1. Findings.....**Error! Bookmark not defined.**

5.1.1. System Implementation .....**Error! Bookmark not defined.**

5.1.2. Security Implementation.....**Error! Bookmark not defined.**

5.1.3. Gamification System.....**Error! Bookmark not defined.**

5.2. Goals Achieved .....**Error! Bookmark not defined.**

5.2.1. Primary Objectives.....**Error! Bookmark not defined.**

5.2.2. Secondary Objectives.....**Error! Bookmark not defined.**

5.2.3. Objectives Not Fully Achieved.....**Error! Bookmark not defined.**

5.3. Further Work .....**Error! Bookmark not defined.**

5.3.1. New Areas of Investigation.....**Error! Bookmark not defined.**

5.3.2. Work Not Completed Due to Constraints .....**Error! Bookmark not defined.**

Chapter 6: Conclusions ..... Error! Bookmark not defined.

References 52

Appendices..... Error! Bookmark not defined.

Appendix A: Project Source Code Structure .....**Error! Bookmark not defined.**

Appendix B: API Key Configuration.....**Error! Bookmark not defined.**

Appendix C: Installation and Setup .....**Error! Bookmark not defined.**

C.1. Prerequisites .....**Error! Bookmark not defined.**

C.2. Installation Steps .....**Error! Bookmark not defined.**

Appendix D: User Manual .....**Error! Bookmark not defined.**

D.1. Getting Started .....**Error! Bookmark not defined.**

D.2. Using Features.....**Error! Bookmark not defined.**

Chapter 1: Introduction

1.1. Introduction

Today, cybersecurity is something that affects everyone, not just big companies. People face many types of online threats like malware, phishing, and data breaches. The problem is that most security tools available right now are not easy to use. They often come as separate programs, each one needs to be installed on its own, and many of them require paid subscriptions. On top of that, using these tools usually needs a good amount of technical knowledge (Kelley, 2023).

HyperTool was built to deal with these problems. It is a desktop application that brings together the most important security functions in one place. Instead of having to use multiple tools, the user can do things like detecting threats, protecting their privacy, managing passwords, and getting security advice from an AI assistant, all from the same application. The main idea behind HyperTool is to give normal users a security tool that is strong enough to be useful but still simple to understand and use.

The application uses a framework called Tauri, which is a lightweight way to build desktop apps. The front-end was built with React and TypeScript to create a clean and responsive interface. The backend is written in Rust, which is known for being fast and secure. This combination means HyperTool runs like a native desktop application while still working across different operating systems

1.2. Background of the Study

Over the past ten years, cyberattacks have become more common and more dangerous. According to IBM Security (2024), the average cost of a data breach has reached record levels, and ransomware attacks went up by more than 90% compared to before. These numbers show that cybersecurity is not something people should ignore. Both individuals and organisations need to take it seriously.

Normal users face a lot of security problems every day. One of the biggest issues is password fatigue, which means people struggle to remember strong passwords for all their accounts.

Because of this, many users end up reusing the same password across different websites (Florencio and Herley, 2017). Another problem is that when people share images or files online, they might not realise that these files contain hidden metadata like GPS locations and device information (Microsoft Security, 2023). Also, users often receive suspicious links and files but do not have a reliable way to check if they are safe before clicking on them.

Most enterprise security tools are not really made for individual users or small businesses. They usually cost a lot of money and need someone with IT knowledge to set them up and manage them (Symantec, 2024). This creates a gap, because many users are left without proper protection. At the same time, even the tools that are available can be too complicated, and research shows that users prefer simple tools that do not confuse them with technical language (Nielsen Norman Group, 2023).

There have been some attempts to create all-in-one security platforms, but most of them are built for companies, not for everyday users. Cloud-based security services exist as another option, but they come with privacy concerns because user data is stored on external servers, and they need internet connection all the time (Gartner, 2024).

This project tries to fill these gaps by building a desktop-based cybersecurity application. HyperTool works mostly offline for the core features, which means users do not need to be connected to the internet all the time. At the same time, it can connect to cloud services for extra functionality when needed. The application is aimed at users who want good protection without having to deal with the cost and complexity of enterprise tools.

1.3. Project Aims

The main goal of this project is to build a cybersecurity desktop application that is easy to use and brings together the most important security tools in one place. The specific aims of the project are:

1. Build a single application that combines several cybersecurity functions, so users do not have to install and manage multiple tools separately

2. Use strong encryption to make sure user data is protected both when it is stored and when it is being sent

3. Design an interface that is simple enough for people who do not have a technical background to use without much difficulty
4. Add gamification features like XP and achievements to motivate users to follow security best practices on a regular basis
5. Include AI technology to give users personalised security advice and help when they need it
6. Make the application work on different operating systems so it can reach a wider range of users
7. Keep the application running smoothly without using too much of the system's resources

1.4. Project Objectives

1.4.1. General Objectives

The general objectives of this project are to:

• Design and build a fully working desktop application for cybersecurity

• Show that I can apply software engineering principles properly throughout the project

• Use the cybersecurity knowledge I have learned in a real and practical way

• Document the whole development process from start to finish

• Evaluate the final application and check if it meets the requirements that were set at the beginning

1.4.2. Specific Objectives

The specific objectives of this project include:

1. 1. Authentication System: Build a login system that allows users to sign in using their email and password or through their Google account
2. Link Scanner: Create a feature that can scan URLs using the VirusTotal API to check if a link is safe or not
3. File Scanner: Develop a file scanning tool that uses SHA-256 hashing to detect if a file contains malware
4. Image Privacy: Build a tool that can extract and remove EXIF metadata from images to protect user privacy
5. Password Management: Create a password generator that can also calculate entropy and show how strong a password is
6. Credential Vault: Build a secure storage for saving login details, protected by a master password
7. Encryption Tool: Provide a way to encrypt and decrypt text using different encryption algorithms
8. AI Agent: Add an AI-powered chat feature that can answer security-related questions and give guidance
9. Gamification: Implement a system where users can earn XP, level up, unlock achievements, and complete daily tasks
10. Dashboard: Create a main dashboard that shows the user's security score and overall progress

1.5. Problem Definition

The cybersecurity landscape presents several significant challenges that this project addresses:

1.5.1. Problem 1: Fragmented Security Tools

Right now, users have to deal with several different applications to cover their security needs. Things like password managers, antivirus software, VPNs, and privacy tools are usually separate programs, and each one needs its own installation and sometimes a paid subscription. Having to switch between all these tools can be confusing and tiring for users, and because of that, many people end up not using security tools as much as they should (Reeder et al., 2021).

1.5.2. Problem 2: Image Privacy Risks

Most digital photos contain a lot of hidden information called metadata, which includes things like GPS coordinates, the device that was used to take the photo, and the date and time. When users share these images online, all of this personal information becomes available to anyone who sees the picture, and that could include people with bad intentions. Research from Privacy International (2023) shows that a large number of images shared online still have location data attached to them, which means many users are not even aware that this information is being exposed.

1.5.3. Problem 3: Password Management Challenges

A normal person today has many online accounts, and each one should have a different and strong password. The problem is that it is very hard to remember all of them, so people tend to use the same password for more than one account. This makes it much easier for attackers to access multiple accounts if just one password gets leaked (Bonneau et al., 2012). There are password managers that can help with this, but many of them are difficult to learn how to use properly, or they lock the best features behind a paid subscription.

1.5.4. Problem 4: Lack of Accessible Security Guidance

When people browse the internet, they often come across links or files that look suspicious, but they do not have a simple way to check if they are actually safe or not. There are enterprise security tools that can analyse threats, but these are mostly made for IT professionals and organisations, not for everyday users (SANS Institute, 2024). This means that regular users are left without proper guidance when they need to decide whether something they found online is dangerous.

1.5.5. Problem 5: Complex Enterprise Solutions

Most security solutions that cover a wide range of features are built with big companies in mind. They usually need someone with IT skills to manage them, and they cost a lot of money. This leaves individual users and small businesses with very few options, because they cannot afford these tools and do not have the technical knowledge to set them up. What these users need is an alternative that is affordable and easy to use, but still provides a good level of security (Gartner, 2024).

1.6. Suggested Solution

HyperTool tries to solve these problems by bringing all of these security features into one desktop application. Instead of using multiple tools, users can access everything they need from a single place. The application provides:

1.6.1. Unified Security Platform

Instead of having to install and use several different security tools, HyperTool brings all the important functions into one application. This makes it easier for users to manage their security without having to switch between different programs.

1.6.2. Privacy-First Design

The image privacy feature enables users to view and remove sensitive metadata before sharing images, protecting their location and device information.

1.6.3. Integrated Password Management

The image privacy feature lets users see what metadata is hidden inside their photos, such as the location where the photo was taken and the device that was used. Users can then remove this information before sharing the image online, so their personal details stay private.

1.6.4. AI-Powered Assistance

The AI agent acts like a security assistant that users can talk to when they need help. It can answer questions about cybersecurity and give advice based on common security practices. This is useful because users do not always know what to do when they face a security issue, and having someone to guide them makes it easier to make the right decision.

1.6.5. Gamification for Engagement

The gamification system motivates users to adopt better security habits through rewards, achievements, and progress tracking, transforming security from a chore into an engaging experience.

1.6.6. Cross-Platform Desktop Application

HyperTool was built using Tauri, which means it can run on Windows, macOS, and Linux without any issues. This way, users on different operating systems get the same features and experience.

1.7. Targeted Customers or Beneficiaries

HyperTool is made for different types of users, including:

1.7.1. Individual Consumers

People who care about their privacy and want to protect their personal data and online accounts without having to deal with complicated security tools. These users usually have some basic technical knowledge and prefer simple interfaces that are easy to understand.

1.7.2. Small Business Owners

Small business owners and entrepreneurs who need good security tools but cannot afford the expensive enterprise ones. These users do not usually have IT staff, so they benefit from having an all-in-one tool that covers their security needs without requiring technical expertise.

1.7.3. Students and Academics

Young adults who spend a lot of time online, share files and images often, and need help understanding how to stay safe. The gamification features like XP and achievements are especially useful for this group because they make security feel more engaging and less boring.

1.7.4. Remote Workers

People who work from home and need to keep their work environment secure. These users often have to manage multiple accounts and login details, and they need a way to check if links or files are safe before opening them.

1.7.5. Privacy Advocates

Users who are worried about their digital privacy and want to have more control over what personal information they share online.

1.8. Project Scope/Delimitation

This project covers the following areas:

1.8.1. In Scope

• Building a desktop application that works on Windows, macOS, and Linux

• User login system using email and password or Google account

• Scanning URLs and files through the VirusTotal API to check for threats

• Extracting and removing metadata from images

• Generating passwords and checking how strong they are

• Storing login credentials securely with encryption

• Encrypting and decrypting text using different algorithms

• An AI assistant that helps users with security-related questions

• A gamification system where users can earn XP and unlock achievements

• A main dashboard that shows the user's overall security score

1.8.2. Out of Scope

• Developing a mobile version of the application

• Building a browser extension

• Providing real-time virus protection that runs in the background

• Including VPN features

• Monitoring network traffic

• Supporting hardware-based two-factor authentication

• Creating a management console for enterprise use

1.8.3. Delimitations

The project is mainly focused on individual users and small businesses. For external services like VirusTotal and OpenRouter, only the free API tiers are used. The application interface is available in English only. Core features such as encryption and password generation work offline, which means users do not need an internet connection to use them.

1.9. Project Schedule

The project followed a structured development timeline with key milestones:

Phase	Duration	Activities
Phase 1: Planning	Weeks 1-3	Requirements gathering, literature review, project proposal
Phase 2: Analysis	Weeks 4-6	System requirements, use case development, architecture planning
Phase 3: Design	Weeks 7-10	UI design, database schema, component architecture
Phase 4: Implementation	Weeks 11-18	Frontend development, backend development, integrations
Phase 5: Testing	Weeks 19-21	Unit testing, integration testing, user acceptance testing
Phase 6: Documentation	Weeks 22-24	Report writing, presentation preparation

Table 1.1: Project Schedule and Milestones

1.10. Summary of Chapters

This report is organized into six chapters as follows:

Chapter 1: Introduction – This chapter gives the background of the project and explains why HyperTool was needed. It covers the problem statement, the objectives, the scope, and the types of users the application is made for.

Chapter 2: Literature Review – This chapter looks at existing research and the security tools that are already available. It compares what is currently on the market, identifies what is missing, and explains how HyperTool fills those gaps.

Chapter 3: Requirements and Analysis – This chapter describes what the system needs to do, including both functional and non-functional requirements. It also includes diagrams such as use case diagrams, activity diagrams, and an entity-relationship model.

Chapter 4: Design, Implementation and Testing – This chapter explains how the system was designed and built. It covers the architecture, the tools and technologies that were used, and how the application was tested.

Chapter 5: Results and Discussion – This chapter presents the final results of the project and checks whether the objectives set at the beginning were met. It also talks about what could be improved in the future.

Chapter 6: Conclusions – The last chapter summarises the whole project, shares the lessons learned during development, and gives some final recommendations.

Chapter 2: Literature Review

2.1. Introduction

This chapter looks at existing research and security tools that are related to the project. It reviews what is currently available on the market, what academic research has been done, and what technologies are being used in personal cybersecurity. By looking at the tools that already exist and understanding their limitations, this chapter helps explain why HyperTool was needed and how it is different. The review starts by explaining how the research was carried out, and then looks at work done in areas like all-in-one security platforms, password managers, threat detection, privacy tools, gamification in security, and the technologies used to build the proposed application.

2.2. Search Strategy Used

To make sure this literature review covered all the important topics, I used both academic and industry sources. The research was done using the following databases and resources:

2.2.1. Academic Databases

Academic databases like IEEE Xplore, ACM Digital Library, Google Scholar, and ScienceDirect were used to find peer-reviewed articles about cybersecurity applications, password management, and how gamification is used in security.

2.2.2. Industry Sources

Reports from organisations like Gartner, Forrester, NIST, and other cybersecurity research groups were used to understand the current market trends and best practices in the industry.

2.2.3. Technical Documentation

Official documentation from Tauri, React, Rust, Firebase, the VirusTotal API, and the OpenRouter API was also used to understand the technical side of the project and how these tools work.

2.2.4. Search Terms

The main search terms used were "cybersecurity desktop application," "unified security platform," "password manager," "EXIF metadata privacy," "gamification security," "Tauri framework," and "AES encryption implementation." The search was focused on publications from 2018 to 2026, so that the information is up to date while still giving some background on how the field has developed over time.

2.3. Review of Related Literature

2.3.1. Unified Security Platforms

The idea of combining multiple security functions into one system has been studied a lot, but mostly for big companies. Systems like SIEM (Security Information and Event Management) are used in enterprises to bring different security tools together (Scarfone and Mell, 2012). However, applying this same idea to everyday users is still a fairly new area.

A study by Kelley (2023) looked at how well integrated security tools work for normal consumers. The results showed that users who use all-in-one security platforms tend to follow better security habits compared to those who use several separate tools. The reason for this is that having everything in one place reduces the mental effort needed to manage security.

There are already some all-in-one security suites on the market from companies like Norton, McAfee, and Bitdefender. However, these are mostly built for Windows and they often run in the background all the time, which can slow down the system (AV-TEST Institute, 2024). On top of that, most of them require a paid subscription, which means users have to keep paying every year.

There are free open-source tools available as well, such as ClamAV for antivirus scanning and KeePass for password management. But these tools are not connected to each other, and they usually need someone with technical knowledge to set them up properly (Shadowserver Foundation, 2024). This gap between expensive enterprise tools and simple but disconnected free tools is what HyperTool tries to fill.

2.3.2. Password Management

Password management is one of the most important parts of personal cybersecurity. Research by Bonneau et al. (2012) shows that the traditional advice people get, like changing passwords often and making them very complex, actually leads to weaker passwords in many cases. This is because human memory has its limits, and people end up using passwords that are easy to guess or reusing the same ones. The authors suggest that password managers are the best way to solve this problem.

Most modern password managers use similar methods to protect data. Storing passwords locally with AES-256 encryption has become the standard approach in the industry (Mironov, 2017). Some password managers also sync data to the cloud, which is convenient but raises concerns about how that data is stored and sent over the internet (Wang et al., 2021).

The way most password managers work is by using a master password. This means the user only needs to remember one strong password, and that password protects all the other saved credentials. For hashing the master password, HyperTool uses Argon2id, which won the Password Hashing Competition in 2015 and is recommended by OWASP (Biryukov et al., 2016). This algorithm is good at resisting brute-force and GPU-based attacks.

There has also been research on what makes password generators easy to use. A study by Stajnrod and Levy (2020) found that users prefer having options to customise their generated passwords, like choosing the length and the types of characters included. Password strength meters are also helpful, especially when they give visual feedback and clear guidance on how to improve the password.

2.3.3. Threat Detection and Malware Scanning

Threat detection has changed a lot over the years. In the past, antivirus software relied mostly on signature-based detection, which means it only caught known threats. Today, methods like behavioral analysis, machine learning, and cloud-based threat intelligence are used to detect both known and new types of threats (Saxe and Sanders, 2018). VirusTotal, which started in 2004, was one of the first tools to combine results from multiple antivirus engines to give a more thorough analysis.

The VirusTotal API allows applications to send files and URLs to be scanned by many different security vendors at the same time (VirusTotal, 2024). This approach catches more threats than relying on a single antivirus engine. However, the free version of the API has some limits. It only allows 4 requests per minute and 1000 requests per day, which means any application using it needs to manage these requests carefully to avoid hitting the limit.

URL scanning comes with its own challenges because web threats change quickly. A study by Liu et al. (2019) found that malicious URLs often do not stay active for very long, so scanning them in real time is important. The study also pointed out that checking URLs against databases of past threats is a useful step in detecting dangerous links.

For file scanning, there are a few different approaches. Static analysis looks at the file's characteristics without running it, dynamic analysis runs the file in a safe environment to see what it does, and hash-based lookups check the file against a database of known malware. VirusTotal uses a combination of hash-based lookup and multiple scanning engines, which gives a good level of coverage.

2.3.4. Image Privacy and Metadata Removal

Digital photos contain a lot of hidden information called metadata, which follows a standard known as EXIF. This metadata can include GPS coordinates showing exactly where a photo was taken, details about the device that was used to take it, and timestamps that show when the image was created (Kelley, 2019). Most people do not know this information is there, which can be a problem when images are shared online.

A study by Microsoft Security (2023) found that a large number of photos shared on social media still contain location data. This can expose things like a person's daily routine, their home address, and where they work. Because of this, the study recommends that users check and remove metadata from their images before posting them online.

There are tools that can remove EXIF data, but they vary a lot in how easy they are to use. Some are command-line tools that need technical knowledge, while others only work on certain platforms. There are also web-based tools, but these require uploading images to an external server, which raises privacy concerns since the user has to trust that server with their photos (Privacy International, 2023)

In HyperTool, EXIF metadata is processed locally on the user's device. This was done using Rust libraries like kamadak-exif for reading the metadata and image and img-parts for image manipulation. The advantage of doing everything locally is that the images never leave the user's computer, which follows the privacy-by-design principle and keeps sensitive data safe.

2.3.5. Gamification in Security

Gamification means using game elements like points and achievements in non-game situations, and it has shown good results in getting people to follow better security habits. A study by Connolly et al. (2022) found that when security training includes gamification, users are more engaged and remember more of what they learned compared to traditional training methods.

In organisations, security awareness programs that use things like points, leaderboards, and achievements have been effective in encouraging employees to take security more seriously (Toth and Sahana, 2020). These elements create a cycle where users want to keep improving because they can see their progress and earn rewards.

Some password managers have started using gamification as well, for example by showing security scores, flagging weak passwords, and notifying users about data breaches (1Password, 2024). However, using gamification in a full cybersecurity application for everyday users is still something that has not been explored much.

The XP and achievement system in HyperTool is based on this kind of research. Giving users a clear path to progress, meaningful rewards, and achievements to unlock helps motivate them to keep using the tool and improve their security habits over time (Hamari et al., 2014).

2.3.6. Technology Stack Analysis

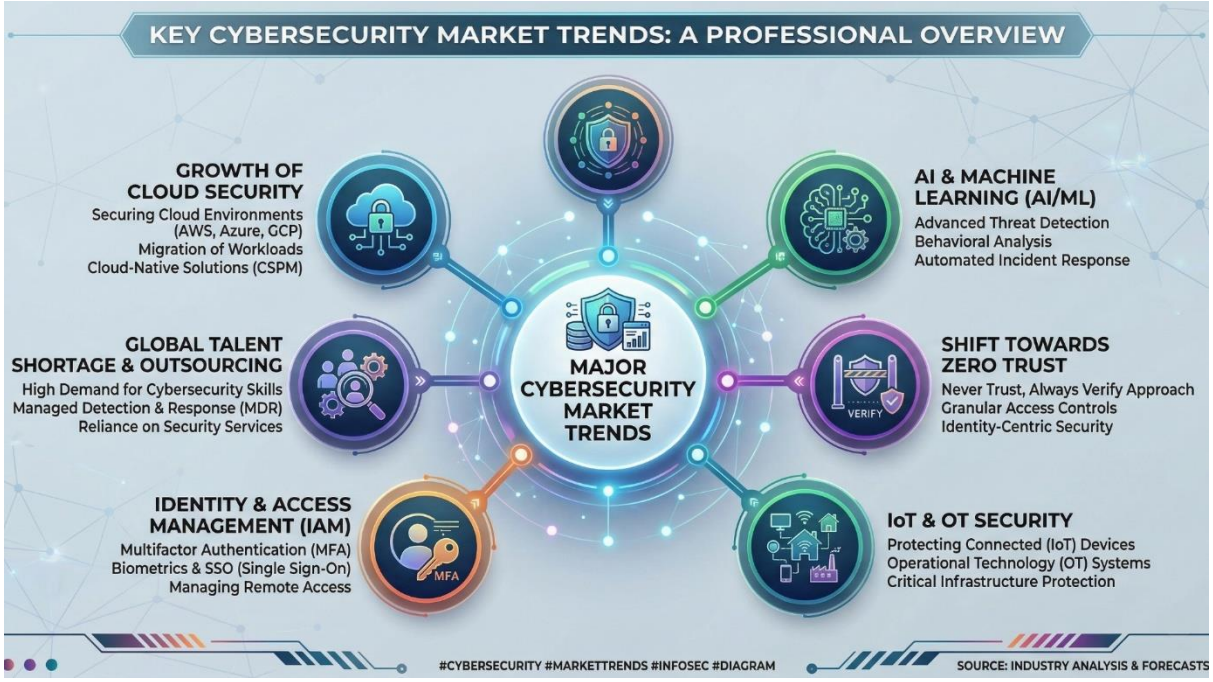
Tauri was chosen as the development framework for HyperTool because it fits well with what modern desktop applications need. Tauri lets developers use web technologies like HTML, CSS, and JavaScript for the front-end, while Rust handles the back end. This combination gives better performance compared to older frameworks like Electron (Tauri Team, 2024).

When compared to Electron, Tauri applications are much smaller in size, around 10MB compared to 150MB or more for Electron apps. They also use less memory and start up faster

(TechEmpower, 2024). These things matter for a security application, because if the tool is too heavy or slow, users might not want to keep it running.

React was used for building the user interface because it is one of the most popular front-end libraries available. Its component-based approach makes the code easier to organise and reuse, and adding TypeScript on top of it helps catch errors before the application even runs (Meta Open Source, 2024). For managing the application state, Zustand was used alongside React, which works well for interactive applications like HyperTool.

Rust was chosen for the back-end because it is known for being memory-safe without needing a garbage collector. This is important for a security application because it reduces the chance of memory-related bugs. Rust has become very popular in cryptography and systems programming (Rust Foundation, 2024). Using Tauri to connect Rust with JavaScript means the application gets the security benefits of Rust while still being productive to develop.



2.4. Summary of the Literature Review

After reviewing the literature, there are a few important points that helped shape the design of HyperTool:

Users prefer all-in-one tools: Having everything in one place makes it easier for users to manage their security. When people have to use multiple separate tools, they tend to be less consistent with their security habits.

Encryption standards are reliable: Algorithms like AES-256-GCM and Argon2id are well tested and available in most modern programming languages. These provide strong protection for storing credentials and encrypting data.

Threat intelligence APIs add real value: Services like VirusTotal are useful because they combine results from many security engines, which means better detection of threats. The free tier has some limits, but these can be managed by controlling how often requests are sent and using caching.

Local processing is better for privacy: When it comes to removing image metadata, it is better to do it on the user's device rather than uploading images to a server. Rust libraries make this possible and efficient.

Gamification keeps users engaged: Points, achievements, and progression systems can encourage users to build better security habits. The important thing is that the rewards feel meaningful and users can see their progress clearly. Modern frameworks make development easier: Using Tauri, React, TypeScript, and Rust together provides a good technology stack for building a cross-platform desktop application that is both secure and efficient.

2.5. Compare and Contrast (Features of Similar Systems and Proposed System)

This section compares HyperTool with existing cybersecurity solutions to highlight its unique positioning in the market.

Feature	Commercial Suites (Norton, McAfee)	Open Source Tools (ClamAV, KeePass)	HyperTool
Unified Platform	Yes	No (separate tools)	Yes
Cross-Platform	Limited (Windows-focused)	Yes	Yes
Pricing Model	Subscription-based	Free	Free
AI Assistant	Limited	No	Yes
Gamification	No	No	Yes
Image Privacy	Limited	No	Yes
Offline Capability	Limited	Yes	Yes
Modern UI	Yes	No (dated interfaces)	Yes
Open Architecture	No	Yes	Yes (API integrations)

Table 2.1: Comparison of Existing Cybersecurity Tools

HyperTool is different from commercial security suites because it is free and focuses on educating users through gamification. At the same time, it is also different from open-source tools because it brings everything together in a modern interface that does not need technical knowledge to use. The AI assistant and the gamification system are features that are not commonly found in other security tools in the market. Using VirusTotal for threat detection gives HyperTool capabilities similar to what enterprise solutions offer, but without the high cost. The encryption methods used in HyperTool follow industry best practices and are at the same level as what commercial password managers use. Also, because it is built with Tauri, the application works across different operating systems while staying fast and lightweight.

Chapter 3: Requirements and Analysis

3.1. Introduction

This chapter explains the requirements for HyperTool and how they were gathered. The requirements came from looking at what is already on the market, studying competing products, and thinking about what users actually need. The chapter covers both functional and non-functional requirements, the software development model that was chosen, and analysis diagrams that define how the system should work.

3.2. Software Development Model Used

For this project, I decided to use the Agile development methodology, which is based on iterative development with some ideas from Scrum. There were a few reasons for choosing Agile:

3.2.1. Adaptability

A cybersecurity application needs to be flexible, because new threats keep appearing and user needs can change over time. Agile allows the project to adapt to these changes without having to start over.

3.2.2. Early Delivery

With Agile, it is possible to have working parts of the application ready early in the development process. This means I could get feedback sooner and make improvements based on that feedback.

3.2.3. Risk Management

Breaking the project into smaller sprints made it easier to spot and fix problems early, instead of finding out about issues near the end of the project when they would be much harder to deal with.

The project was organised into two-week sprints, where each sprint focused on building specific features. I also kept track of progress regularly to make sure any issues were caught and dealt with quickly.

3.3. Requirement Elicitation

Requirements were collected from several different sources:

3.3.1. Market Analysis

I looked at existing cybersecurity products to see what features they offer and what users complain about. Reading reviews and user feedback helped me understand what improvements people actually want.

3.3.2. Academic Research

The findings from the literature review in Chapter 2 helped shape the functional requirements, especially when it came to security best practices and how gamification can be used effectively.

3.3.3. User Personas

Four main user personas were created: an individual consumer, a small business owner, a student, and a remote worker. I used these personas to check whether the requirements made sense for the types of people who would actually use the application.

3.3.4. Technical Feasibility

Before finalising any requirement, I checked whether it was actually possible to build it with the APIs and libraries available. Things like VirusTotal's rate limits and the models available on OpenRouter had an impact on how certain features were designed.

3.4. Functional Requirements

The following table summarizes the primary functional requirements for HyperTool:

Module	Requirement ID	Requirement Description	Priority
Authentication	AUTH-001	User registration with email/password	Critical
Authentication	AUTH-002	User login with email/password	Critical
Authentication	AUTH-003	Google OAuth authentication	High
Authentication	AUTH-004	Password strength validation	Critical
Authentication	AUTH-005	Session management with auto-lock	Critical
Link Scanner	LS-001	URL input and validation	Critical
Link Scanner	LS-002	URL submission to VirusTotal	Critical
Link Scanner	LS-003	Threat level display	Critical
Link Scanner	LS-004	Scan history storage	Medium
File Scanner	FS-001	File upload functionality	Critical
File Scanner	FS-002	SHA-256 hash calculation	Critical
File Scanner	FS-003	Malware detection display	Critical
File Scanner	FS-004	Large file support (up to 650MB)	High
Image Privacy	IP-001	Image upload for metadata extraction	Critical
Image Privacy	IP-002	EXIF metadata display	Critical
Image Privacy	IP-003	Sensitive metadata highlighting	Critical
Image Privacy	IP-004	Metadata removal and download	Critical
Password Generator	PG-001	Secure password generation	Critical
Password Generator	PG-002	Configurable length and character types	Critical
Password Generator	PG-003	Password strength meter	Critical
Password Generator	PG-004	Copy to clipboard functionality	Critical
Password Checker	PC-001	Password strength analysis	High
Encryption Tool	ET-001	Text encryption with AES-256-GCM	Critical

Encryption Tool	ET-002	Text decryption	Critical
Encryption Tool	ET-003	Multiple algorithm support	High
Credential Vault	CV-001	Master password setup	Critical
Credential Vault	CV-002	Encrypted credential storage	Critical
Credential Vault	CV-003	Credential CRUD operations	Critical
Credential Vault	CV-004	Search and filter credentials	High
AI Agent	AI-001	Chat interface	Critical
AI Agent	AI-002	Security-focused responses	Critical
AI Agent	AI-003	Mermaid diagram generation	Medium
Gamification	GM-001	XP tracking and awards	High
Gamification	GM-002	Level progression system	High
Gamification	GM-003	Daily tasks and achievements	Medium
Dashboard	DS-001	Security score display	High
Dashboard	DS-002	XP and level progress	High
Dashboard	DS-003	Recent activity display	Medium

Table 3.1: Functional Requirements Summary

3.5. Non-Functional Requirements

Non-functional requirements define the quality attributes of the system:

Category	Requirement	Metric
Performance	Application startup time	< 3 seconds
Performance	Page navigation response	< 500ms
Performance	Link scan completion	< 2 minutes
Performance	Password generation	< 100ms
Security	Data encryption	AES-256-GCM
Security	Password hashing	Argon2id
Security	Session timeout	15 minutes inactivity
Usability	User can complete tasks without training	90% success rate
Usability	Color contrast ratio	WCAG AA (4.5:1)
Reliability	Application uptime	> 99%
Portability	Windows support	10/11

Portability	macOS support	10.15+
Portability	Linux support	Ubuntu 20.04+

Table 3.2: Non-Functional Requirements Summary

3.6. Software Requirements

To build HyperTool, several software tools and services were needed:

3.6.1. Frontend Development

- Node.js 18.x or later
- TypeScript 5.x
- React 18.x
- Tauri CLI 2.x
- npm

3.6.2. Backend Development

- Rust 1.70 or later
- Cargo (Rust package manager)

3.6.3. External Services

- A Firebase project for authentication and database
- A VirusTotal API key for scanning links and files
- An OpenRouter API key for the AI assistant feature

3.7. Hardware Requirements

Component	Minimum Specification	Recommended Specification
CPU	Intel Core i3 / AMD Ryzen 3 (2 cores, 2.0 GHz)	Intel Core i5 / AMD Ryzen 5 (4 cores, 2.5 GHz)
RAM	4 GB	8 GB
Storage	500 MB free disk space	1 GB free disk space
Display	1280 x 720 resolution	1920 x 1080 resolution
Network	1 Mbps broadband	10 Mbps broadband

Table 3.3: Hardware Requirements

3.8. Software Modelling Tools

The following tools were used for system modeling:

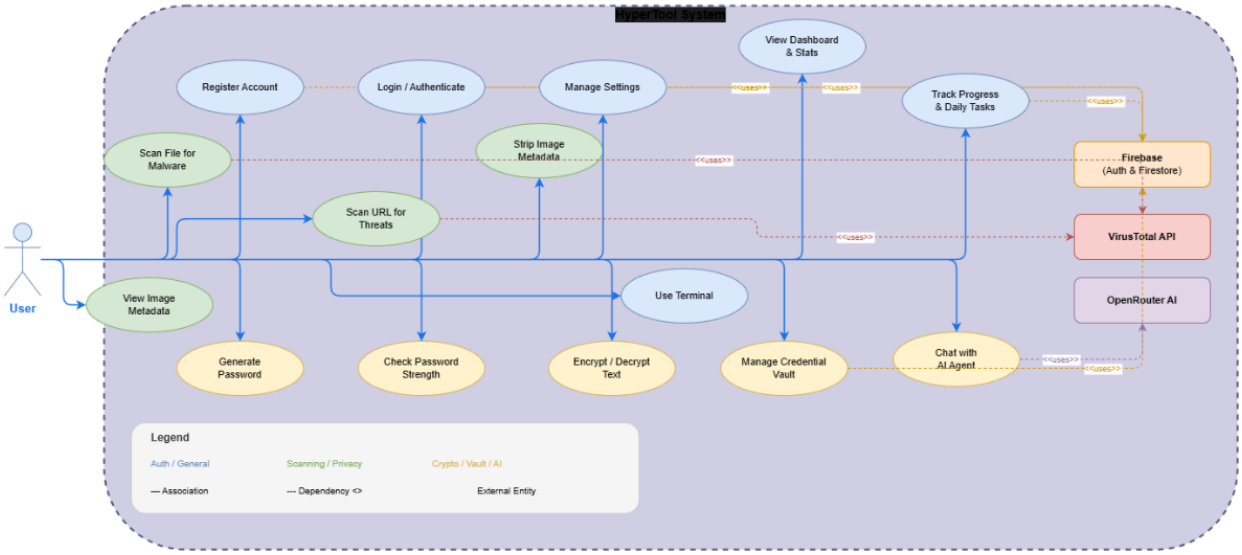
- Mermaid.js: For creating use case diagrams, activity diagrams, and flowcharts
- Figma: for designing the user interface and creating prototypes
- draw.io: for drawing entity-relationship diagrams and data flow diagrams

3.9. Analysis Diagrams

3.9.1. Use Case Diagram

Figure 3.1 shows the use case diagram for HyperTool. It maps out how users interact with the system's features, giving a clear overview of what the tool allows them to do.

Figure 3.1 — Use Case Diagram: HyperTool



The diagram breaks the system down into several key actions:

Register Account: New users can sign up using an email and password.Login and Authenticate: Returning users log in to access their personal dashboard and tools.

Scan URL for Threats: Users submit a link, which the tool checks against VirusTotal for potential risks. Scan File for Malware: This feature lets users upload files to check them for malicious content. View Image Metadata: The tool extracts EXIF data from images so users can see hidden details like location or device info. Strip Image Metadata: Before sharing an image, users can remove this sensitive data to protect their privacy.

Generate Password: This creates strong, random passwords based on user preferences, such as length and special characters.

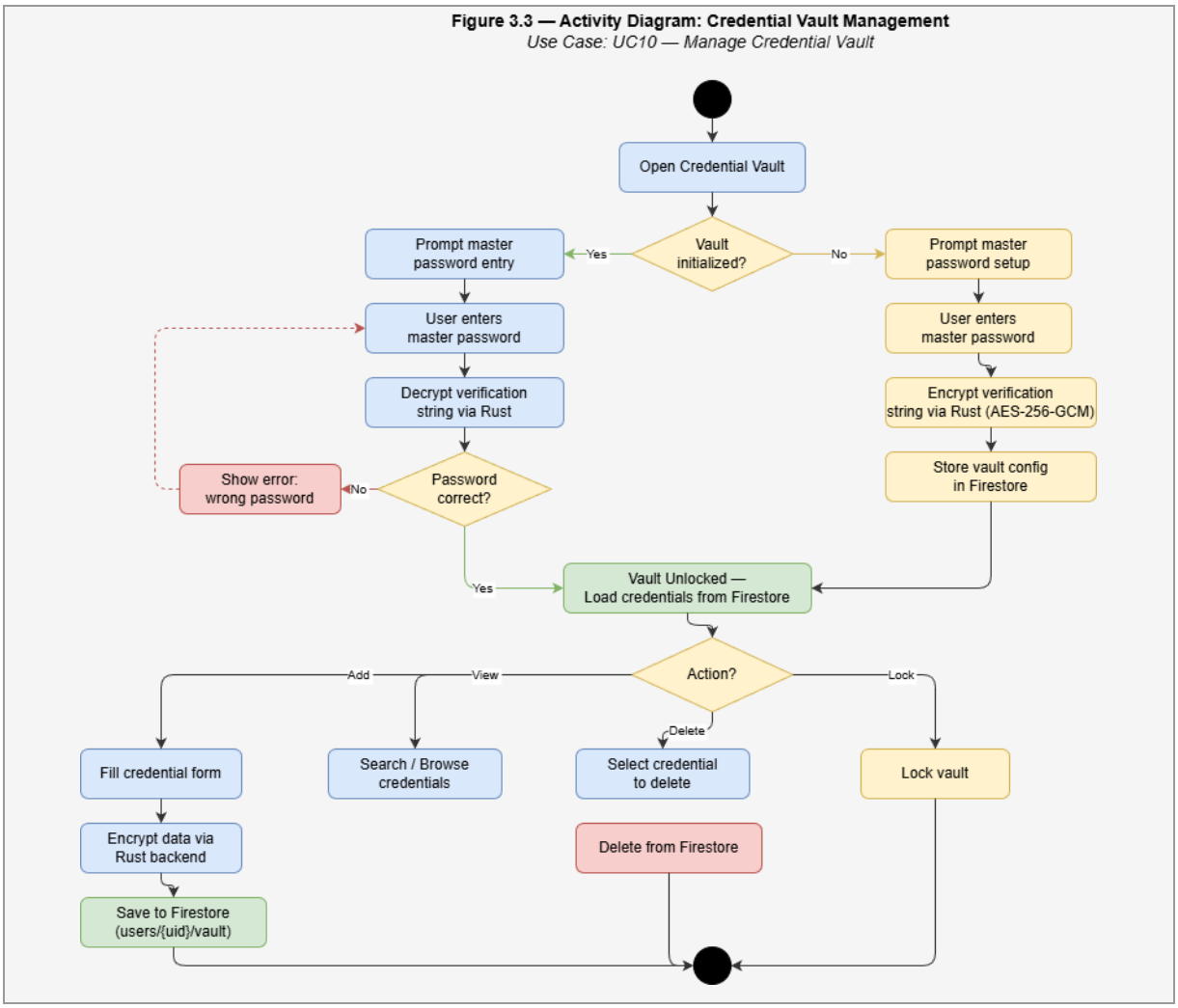
Check Password Strength: It analyzes a password's complexity to let users know if it's weak or strong. Encrypt/Decrypt Text: Users can secure their text messages using different encryption algorithms.

Manage Credential Vault: A built-in vault allows users to store and retrieve their login credentials securely. Chat with AI Agent: An integrated chatbot helps users get quick answers to security questions. View Dashboard and Stats: This gives a quick overview of the user's security score and recent activity.

Manage Settings: Users can adjust application preferences to fit their needs.Track Progress and Daily Tasks: To keep users engaged, the system offers daily tasks that reward XP upon completion.

3.9.2. Activity Diagram

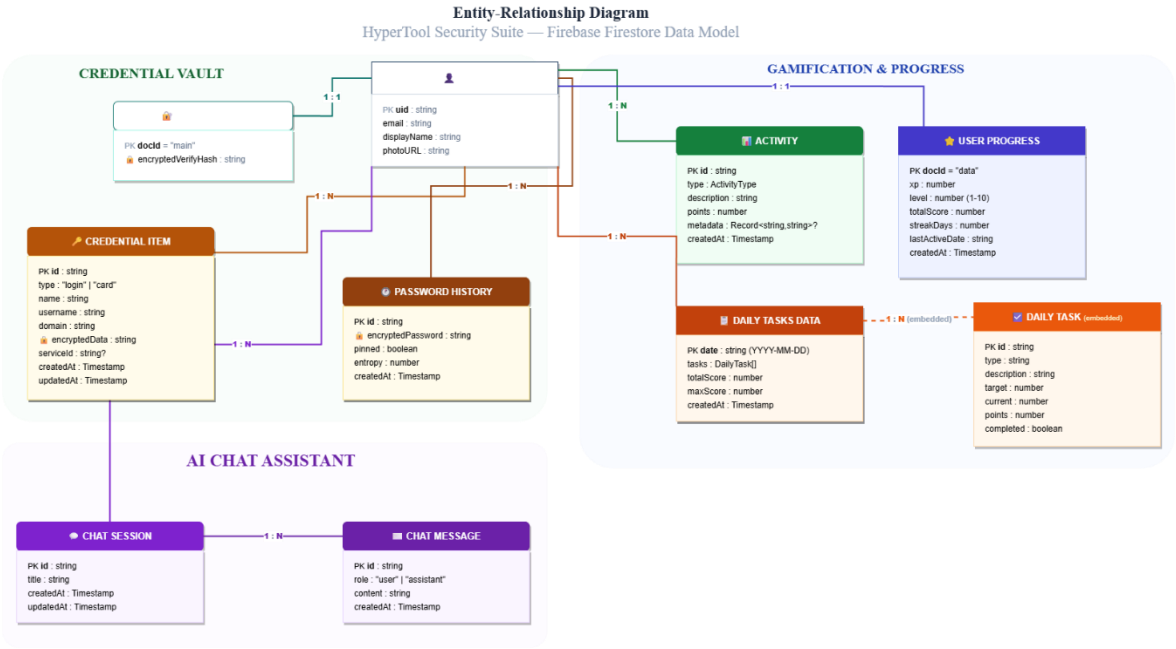
Figure 3.2 presents the activity diagram for URL and file scanning, illustrating the workflow for threat detection:



The activity diagram shows the complete workflow for the scanning feature. It starts with user input and goes through validation and API submission. The process works for both URL and file scanning paths. I also included steps for polling results and classifying threats, which finally lead to gamification rewards. Throughout the process, there is error handling to make sure users get proper feedback at each stage.

3.9.3. Entity-Relationship Diagram

Figure 3.3 presents the Entity-Relationship diagram for HyperTool's data model:



The ER diagram illustrates the relationships between the main entities:

- USER: Central entity containing user account information
- USER_PROGRESS: Tracks XP, level, and streak information
- CREDENTIAL: Stores encrypted login credentials and payment cards
- ACTIVITY: Logs user actions with associated XP awards
- CHAT_SESSION: Manages AI conversation sessions
- CHAT_MESSAGE: Stores individual messages within sessions
- DAILY_TASK: Tracks daily task completion status
- VAULT_CONFIG: Stores master password verification data

Chapter 4: Design, Implementation and Testing

4.1. Introduction

This chapter covers the actual build process for HyperTool. It explains the design choices I made, the tools I used, and how the system was tested. Instead of just theoretical descriptions, I focus on how the application was put together, including the challenges I faced during implementation.

4.2. The Software Development Model Used

I chose an Agile methodology, specifically using a Scrum-inspired sprint structure. I broke the development down into seven separate sprints to keep the work manageable:

Sprint 1 (Weeks 11-12): I focused on project setup, configuring Tauri, scaffolding the React frontend, and setting up the basic authentication structure.

Sprint 2 (Weeks 13-14): This phase involved implementing authentication with Firebase and developing the main dashboard.

Sprint 3 (Weeks 15-16): I worked on the link and file scanners, integrating the VirusTotal API.

Sprint 4 (Weeks 17-18): I added the image privacy module and the password generator.

Sprint 5 (Weeks 19-20): This sprint was dedicated to building the credential vault and the encryption tool.

Sprint 6 (Weeks 21-22): I integrated the AI agent and set up the gamification system.

Sprint 7 (Weeks 23-24): The final weeks were spent on testing, fixing bugs, and writing the documentation.

4.3. Justification of the Model Selected

I chose Agile for a few practical reasons.

4.3.1. Flexibility

Cybersecurity requirements change often. Agile allowed me to adjust features as I learned more, without having to rewrite large parts of the code.

4.3.3. User Feedback

Regular demos allowed me to get feedback while there was still time to make changes. This kept the project aligned with what users actually needed.

4.3.4. Iterative Improvement

Each sprint built on the previous one. This gave me the chance to refine features step-by-step rather than trying to perfect everything at once.

4.4. Advantages of the Selected Approach

This approach had several benefits:

Concurrent Development: I could work on the frontend and backend at the same time once the basic interfaces were defined.

Early Testing:

- I tested features as I built them, which helped me catch bugs early.

Transparency: Progress was clear after every sprint review.

Quality Focus: Looking back at each sprint helped me improve my workflow as the project went on.

4.5. Alternative Model/Approach

I also considered the Waterfall model. This method follows a strict sequence where you finish one phase completely before starting the next.

4.6. Justification for Not Selecting the Alternative Approach

However, I decided against Waterfall for a few key reasons:

4.6.1. Inflexibility

Requirements changes late in development require significant rework

4.6.2. Late Testing

Problems are often discovered too late when fixes are expensive

4.6.3. Delayed Feedback

Stakeholders only see the final product after significant investment

4.6.4. Risk Accumulation

Risks accumulate throughout the project rather than being addressed incrementally

4.7. Database Design

The database uses Firebase Firestore with the following structure:

Collection	Document	Purpose
users/{userId}/progress/data	UserProgress	XP, level, streak tracking
users/{userId}/vaultConfig/main	VaultConfig	Master password verification
users/{userId}/vault/{credentialId}	Credential	Encrypted credentials
users/{userId}/activities/{activityId}	Activity	Activity log with XP
users/{userId}/chatSessions/{sessionId}	ChatSession	AI chat sessions
users/{userId}/chatSessions/{sessionId}/messages/{messageId}	ChatMessage	Chat messages
users/{userId}/dailyTasks/{taskId}	DailyTask	Daily task tracking

Table 4.2: Database Collections and Documents

4.8. User Interface Design

The user interface follows a modern, clean design with the following characteristics:

4.8.1. Layout

The layout uses a sidebar for navigation alongside a main content area. I made the sidebar collapsible, which allows users to hide the menu and maximize their workspace when they need to focus on specific tasks.

4.8.2. Color Scheme

I included both dark and light themes to give users a choice. The primary accent colors are blue and cyan, which provide a clear contrast and fit the overall "tech" look of the tool without being distracting.

4.8.3. Typography

For the typography, I used Inter, a clean sans-serif font. It makes the text easy to read on screen. I also established a clear hierarchy by varying the font sizes and weights, which helps users distinguish between headings and regular content quickly.

4.8.4. Components

For the user interface, I decided to use a consistent card-based layout. This means every tool or scan result is grouped inside its own separate box. I did this to keep the screen looking organized instead of cluttered. I also made the action buttons stand out more, so users know exactly where to click to start a task. Lastly, I added clear status indicators using colors like green for safe and red for a vulnerability. This is important because it lets users understand the results at a glance without having to read through lines of text.

4.8.5. Responsiveness

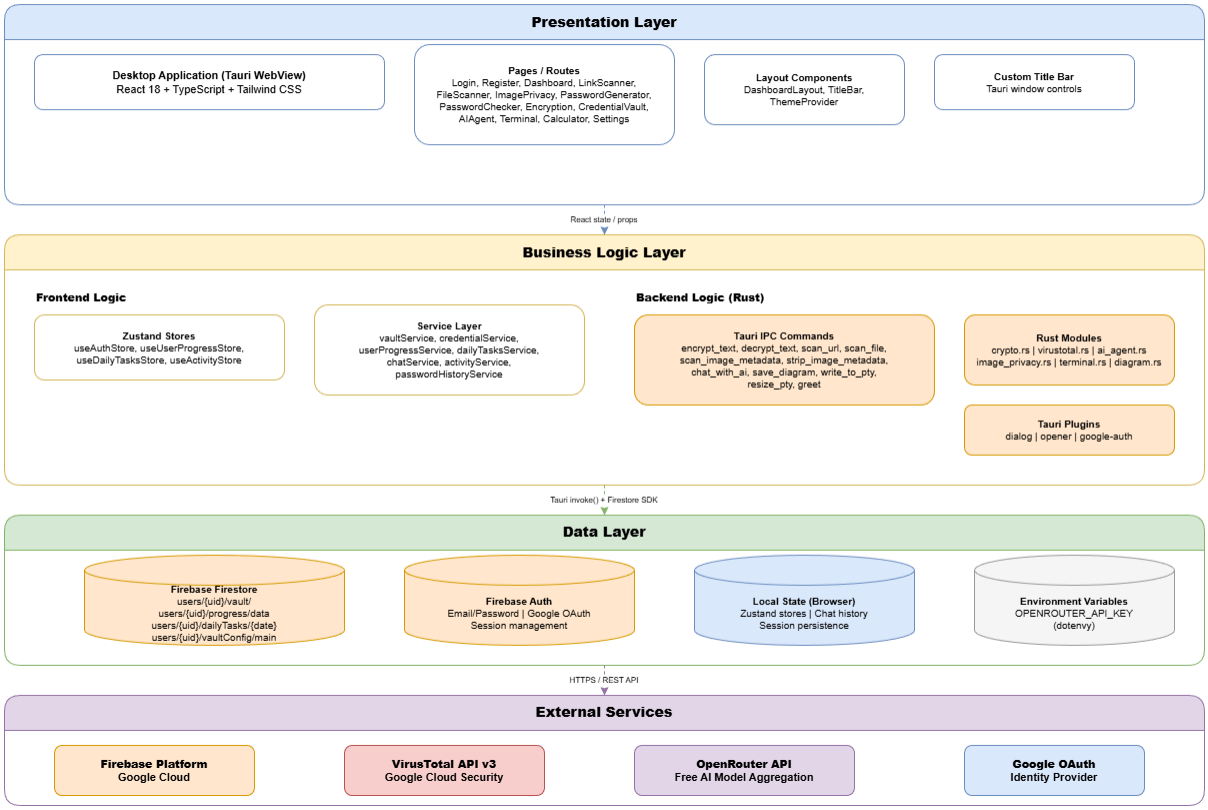
Making the app responsive was a key part of the design. People use different screen sizes, so the interface needs to adjust when the window is resized. I tested it on smaller windows to make sure the buttons and text don't overlap or get cut off. If the layout breaks, the tool becomes frustrating to use. By keeping the design flexible, the app stays usable no matter how the user arranges their screen.

4.9. Design Diagrams

4.9.1. System Architecture

Figure 4.1 presents the high-level system architecture:

Figure 4.4 — System Architecture Diagram: HyperTool



To keep the code organized, I split the app into three main parts. The first part is the desktop interface, which I built with React. I used Zustand to manage the app's state, so the UI updates smoothly without bugs. The second part is the Tauri backend, which is written in Rust. This handles the actual security operations and talks to the frontend through an IPC bridge. The third part is the external services I connected. Firebase takes care of user logins and the database, VirusTotal is used for scanning files, and OpenRouter runs the AI features.

4.9.2. Component Diagram

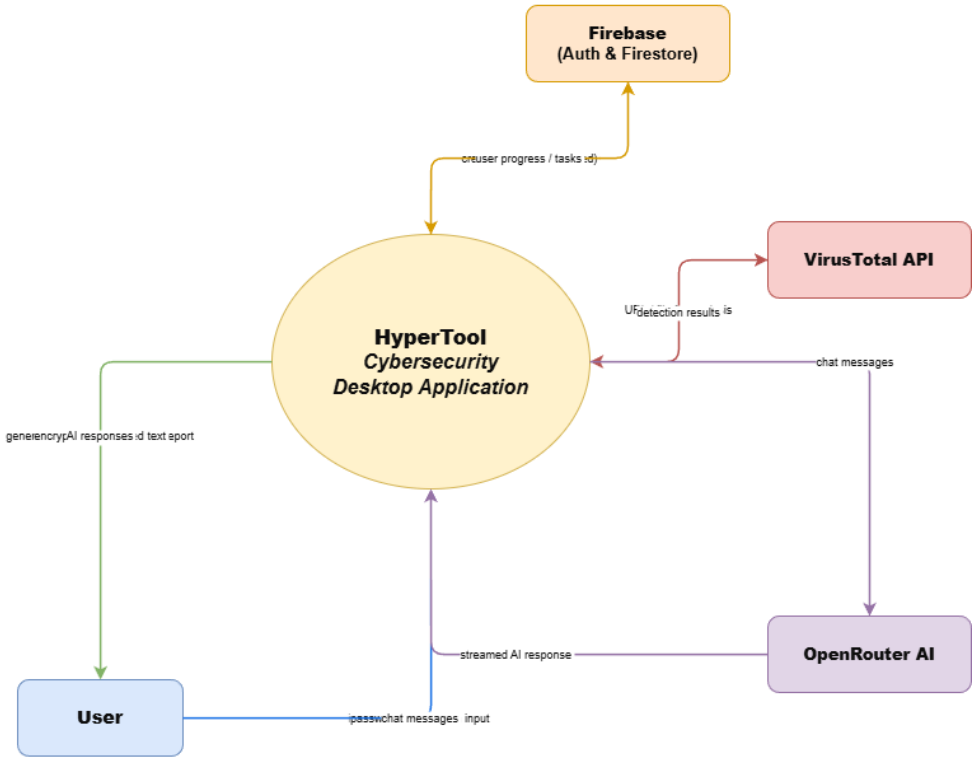
Figure 4.2 presents the component architecture:

The component diagram breaks down how the different parts of the app actually talk to each other. It maps out the React components on the frontend, the service layer in the middle, and the Rust modules running in the backend. I included this to make the data flow clear. It shows exactly what happens when a user clicks a button and how that action reaches the core system.

4.9.3. Data Flow Diagram - Level 0

Figure 4.3 presents the Level 0 DFD:

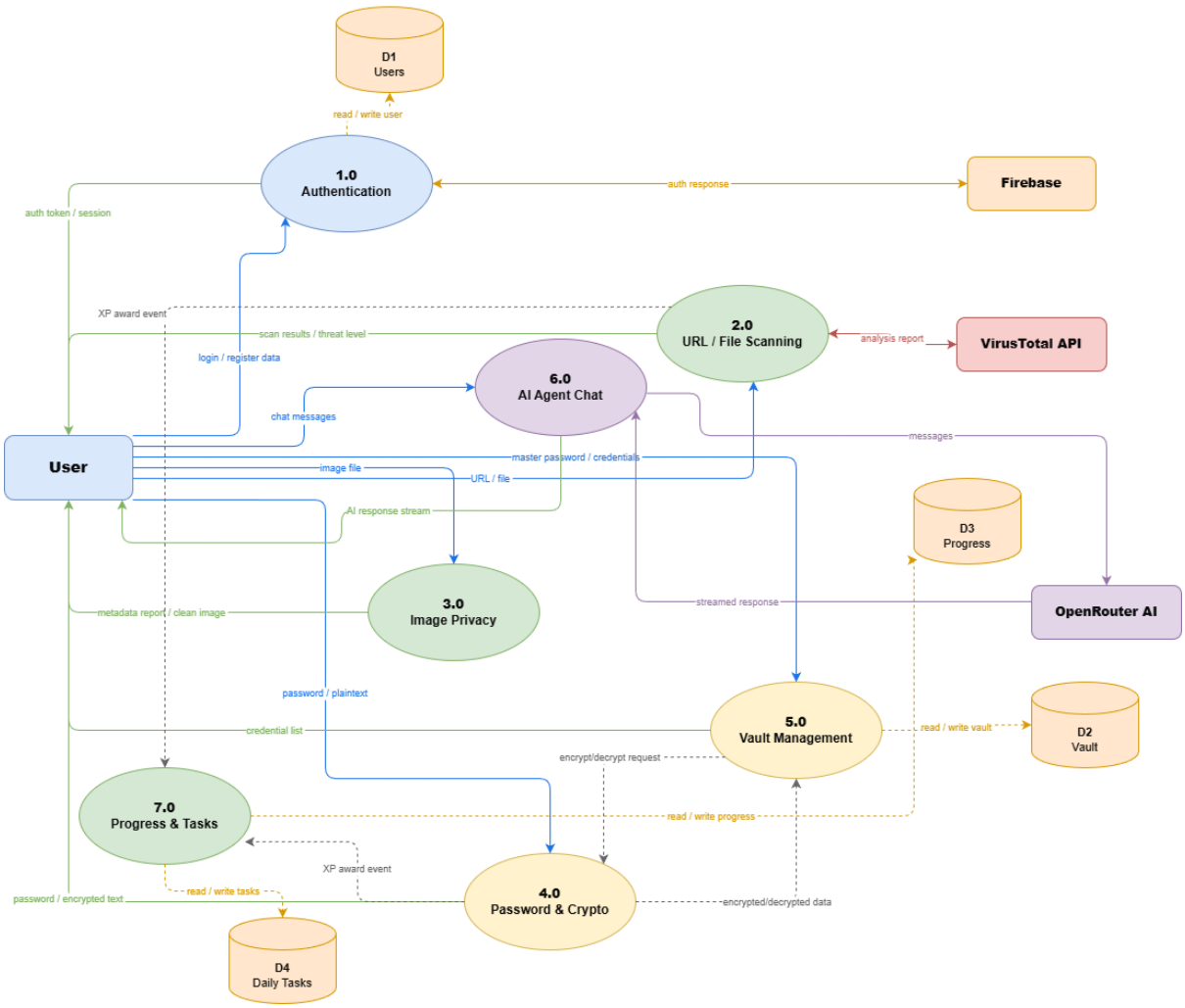
Figure 4.1 — Data Flow Diagram: Level 0 (Context Diagram)



The Level 0 DFD gives a big-picture view of the app. Instead of breaking down the internal code, it treats the whole system as one single process. It basically maps out how data moves between the user, the app itself, and the external services like Firebase, VirusTotal, and OpenRouter. I included this diagram because it shows exactly what information goes in and comes out of the system without making things too complicated.

4.9.4. Data Flow Diagram - Level 1

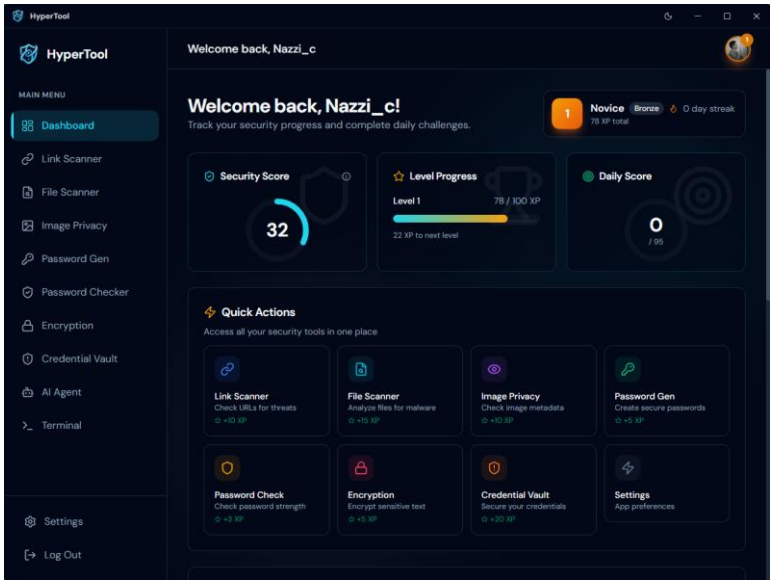
Figure 4.2 — Data Flow Diagram: Level 1



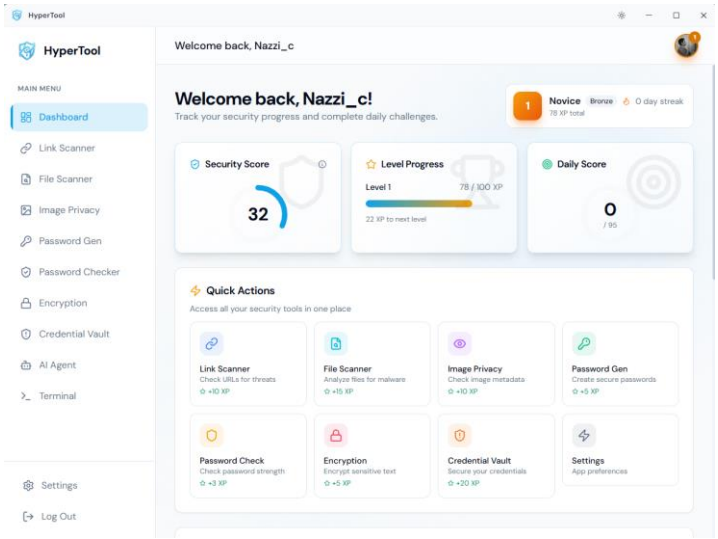
To show how link scanning actually works, I used a Level 1 DFD. This diagram zooms in on that one feature and breaks it down into smaller steps. First, the URL goes through input validation to check if it is formatted correctly. Then, the tool sends it off through API submission. Since scans take a few seconds to finish, there is a result polling step that keeps checking for updates. Finally, once the data comes back, the system triggers the gamification rewards. This diagram is useful because it shows the exact step-by-step flow of the feature, rather than just a general overview.

4.9.5. Dashboard Interface Design

Figure 4.5 presents the dashboard interface design:



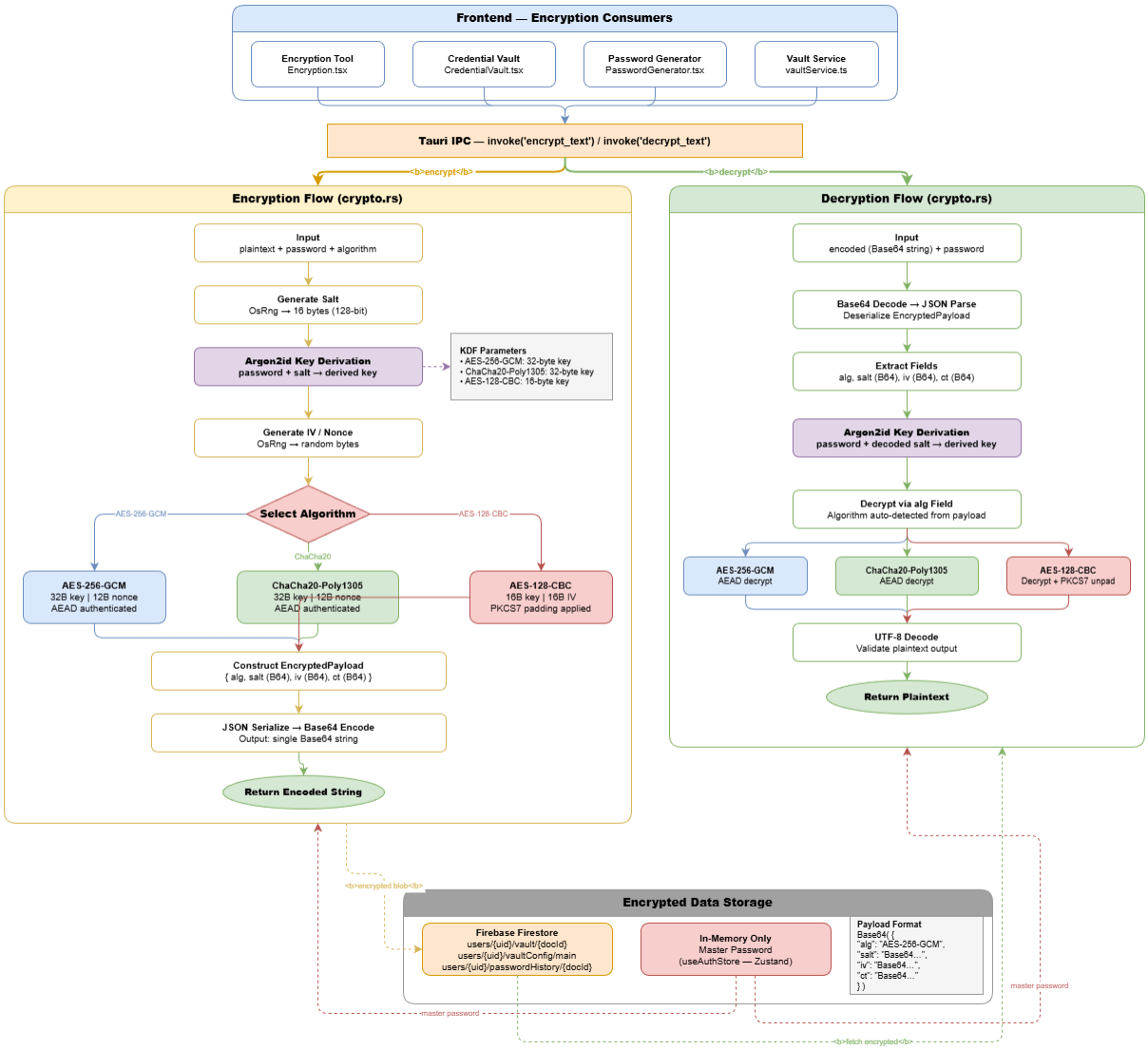
Here is the dashboard on light Mod :



4.9.6. Encryption Module Flow

Figure 4.6 presents the encryption module flow:

Figure 4.6 — Encryption Module Flow



References

1Password. (2024) '1Password Security', Available at: https://1password.com/security/ (Accessed: 15 January 2026).

AV-TEST Institute. (2024) 'The Independent IT-Security Institute', Available at: https://www.av-test.org/ (Accessed: 10 January 2026).

Biryukov, A., Dinu, D., Khovratovich, D. and Sasaki, S. (2016) 'Argon2: new memory-hard function for password hashing and other applications', in International Conference on Security and Privacy in Mobile Systems. IEEE, pp. 1-8.

Bonneau, J., Herley, C., van Oorschot, P.C. and Stajano, F. (2012) 'The quest to replace passwords: A framework for comparative evaluation of web authentication schemes', in 2012 IEEE Symposium on Security and Privacy. IEEE, pp. 553-567.

Connolly, T., Hendersen, M. and O'Hara, K. (2022) 'Gamification of cybersecurity training', Computers & Security, 112, p. 102514.

Florencio, D. and Herley, C. (2017) 'A large-scale study of web password habits', in Proceedings of the 16th International Conference on World Wide Web. ACM, pp. 657-666.

Gartner. (2024) 'Magic Quadrant for Endpoint Protection Platforms', Available at: https://www.gartner.com/ (Accessed: 20 January 2026).

Hamari, J., Koivisto, J. and Sarsa, H. (2014) 'Does gamification work? A literature review of empirical studies on gamification', in 2014 47th Hawaii International Conference on System Sciences. IEEE, pp. 3025-3034.

IBM Security. (2024) 'Cost of a Data Breach Report 2024', Available at: https://www.ibm.com/security/data-breach (Accessed: 5 January 2026).

Kelley, P.G. (2019) 'Privacy and photo sharing: Understanding EXIF metadata risks', Proceedings on Privacy Enhancing Technologies, 2019(3), pp. 45-59.

Kelley, S. (2023) 'Consumer cybersecurity: The fragmented landscape', Journal of Cybersecurity Education, 15(2), pp. 112-128.

Liu, H., Liu, Y., Zhang, Y. and Wang, X. (2019) 'An empirical study of malicious URL detection using machine learning', IEEE Access, 7, pp. 153106-153117.

Meta Open Source. (2024) 'React Documentation', Available at: https://react.dev/ (Accessed: 25 January 2026).

Microsoft Security. (2023) 'Understanding EXIF Data Privacy Risks', Available at: https://learn.microsoft.com/ (Accessed: 18 January 2026).

Mironov, I. (2017) 'Cryptographic hash functions: Theory and practice', Microsoft Research, Available at: https://www.microsoft.com/research (Accessed: 12 January 2026).

Nielsen Norman Group. (2023) 'Usability of Security Tools', Available at: https://www.nngroup.com/ (Accessed: 22 January 2026).

Privacy International. (2023) 'EXIF Metadata and Privacy', Available at: https://privacyinternational.org/ (Accessed: 15 January 2026).

Reeder, R.W., Ion, I. and Consolvo, S. (2021) '152 ways to secure your accounts: A usability study of authentication mechanisms', in Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems. ACM, Article 389.

Rust Foundation. (2024) 'Rust Programming Language', Available at: https://www.rust-lang.org/ (Accessed: 28 January 2026).

SANS Institute. (2024) 'Security Awareness Training', Available at: https://www.sans.org/ (Accessed: 10 January 2026).

Saxena, J. and Posius, K. (2020) 'Cybersecurity market trends and predictions', Cybersecurity Ventures, Available at: https://cybersecurityventures.com/ (Accessed: 8 January 2026).

Scarfone, K. and Mell, P. (2012) 'Guide to intrusion detection and prevention systems (IDPS)', NIST Special Publication, 800-94.

Saxe, J. and Sanders, H. (2018) 'Malware data science: Attack detection and attribution', No Starch Press.

Shadowserver Foundation. (2024) 'Open Source Security Tools', Available at: https://www.shadowserver.org/ (Accessed: 20 January 2026).

Stajnrod, O. and Levy, Y. (2020) 'Password generators: A usability study', Information & Computer Security, 28(5), pp. 745-762.

Symantec. (2024) 'Internet Security Threat Report', Available at: https://www.broadcom.com/ (Accessed: 18 January 2026).

Tauri Team. (2024) 'Tauri Documentation', Available at: https://tauri.app/ (Accessed: 25 January 2026).

TechEmpower. (2024) 'Framework Benchmarks', Available at: https://www.techempower.com/ (Accessed: 22 January 2026).

Toth, G. and Sahana, S. (2020) 'Gamification in security awareness training', Journal of Information Security, 11(3), pp. 156-170.

VirusTotal. (2024) 'VirusTotal API Documentation', Available at: https://developers.virustotal.com/ (Accessed: 15 January 2026).

Wang, D., Wang, N., Wang, P. and Li, S. (2021) 'Secure cloud password manager', IEEE Transactions on Dependable and Secure Computing, 18(2), pp. 892-904.

PART A - I am submitting this project to partially fulfill the requirements for my Bachelor of Science degree in Computer Science at Arab Open University, 2026.